pandas: der Datenworkflow

M2 · Data-Science-Werkzeugkasten

Die Beispieltabelle small

Alle Beispiele dieser Karte laufen auf einer kleinen Tabelle mit fünf Passagieren, die Sie in einer Zelle anlegen und daneben ausprobieren können. Jeder Abschnitt hat denselben Aufbau: ein Satz, dann eine Tabelle **Absicht, Code, Ergebnis**, zuletzt eine typische Falle. „liefert“ heißt: Es entsteht ein neues Objekt, die Tabelle bleibt unverändert; übernommen wird mit `small = small.dropna()`. „ändert“ heißt: Die Tabelle selbst wird verändert. Die Ergebnisangaben beziehen sich auf die ursprüngliche Tabelle `small`; Änderungen probieren wir an einer Kopie `beispiel` aus, damit `small` für alle Beispiele gleich bleibt.

```
import numpy as np
import pandas as pd
small = pd.DataFrame({"pclass": [1, 1, 3, 3, 2],
                      "sex": ["female", "male", "male", "female", "male"],
                      "age": [29.0, 0.75, np.nan, 18.0, 54.0]})
```

small Form (5, 3); dtypes int64, str, float64

	pclass	sex	age
0	1	female	29.0
1	1	male	0.75
2	3	male	NaN
3	3	female	18.0
4	2	male	54.0

`small["age"]` →

Series Form (5,), float64

	age
0	29.0
1	0.75
2	NaN
3	18.0
4	54.0

`.to_numpy()` →

Array

0	1	2	3	4
29.0	0.75	NaN	18.0	54.0

Form (5,), float64; nur Positionen, keine Etiketten
sub = small[small["pclass"] == 3] behält die Etiketten 2 und 3 an den Positionen 0 und 1: loc fragt nach dem Etikett, iloc nach der Position.

Ein `DataFrame` ist eine Sammlung benannter Spalten. Jede Spalte ist eine `Series` mit genau einem Datentyp. Der Index gibt jeder Zeile ein Etikett (hier die Nummern 0 bis 4), alle Spalten teilen ihn. Ein Etikett bleibt an seiner Zeile, auch nach einem Filter; die Position zählt dagegen im Ergebnis neu von 0. Textspalten heißen unter pandas 3 `str`, unter älteren Versionen `object`.

Einlesen und Überblick gewinnen

`read_csv` erkennt den Datentyp jeder Spalte selbst: nur ganze Zahlen `int64`, mit Nachkommastellen oder Lücken `float64`, ein einziger Nicht-Zahl-Eintrag macht die Spalte zu Text.

Absicht	Code	Ergebnis
CSV-Datei lesen	`pd.read_csv(pfad, encoding="utf-8")`	DataFrame
`?` als Lücke lesen	`pd.read_csv(pfad, na_values=["?"])`	Spalte wird `float64` statt Text
Leerzeichen aus Spaltennamen	`small.columns = small.columns.str.strip()`	ändert die Namen (hier gibt es keine)
erste Zeilen ansehen	`small.head(3)`	die ersten drei Zeilen
Form	`small.shape`	`(5, 3)`
Typ je Spalte	`small.dtypes`	`int64`, `str`, `float64`
Anzahl und Typ je Spalte	`small.info()`	Übersicht mit Lücken
Kennzahlen einer Zahlenspalte	`small["age"].describe()`	Anzahl 4, Mittelwert 25.44, …
Lücken je Spalte	`small.isna().sum()`	`age 1`, sonst 0

Falle: Steht in einer Zahlenspalte ein `?` oder ein Wort, liest `read_csv` die ganze Spalte als Text. `na_values` oder `pd.to_numeric(s, errors="coerce")` (ungültige Einträge werden Lücken) beheben das.

Spalten auswählen

Ein Name in eckigen Klammern liefert eine Series, eine Liste von Namen einen DataFrame.

Absicht	Code	Ergebnis
eine Spalte	`small["age"]`	Series, Form `(5,)`
eine Spalte als Tabelle	`small[["age"]]`	DataFrame, Form `(5, 1)`
mehrere Spalten	`small[["pclass", "age"]]`	DataFrame, Form `(5, 2)`
Spaltennamen	`small.columns`	`pclass, sex, age`
Spalten entfernen	`small.drop(columns=["age"])`	liefert Tabelle `(5, 2)`

Falle: `small["pclass", "age"]` ohne innere Klammern ist ein `KeyError`; pandas sucht eine Spalte mit dem Namen `("pclass", "age")`.

Sortieren und Zählen

Absicht	Code	Ergebnis
nach Spalte ordnen	`small.sort_values("age")`	liefert Tabelle, aufsteigend, `NaN` zuletzt
absteigend	`small.sort_values("age", ascending=False)`	54.0 zuerst
die größten n	`small.nlargest(2, "age")`	Zeilen 4 und 0
Häufigkeit je Wert	`small["sex"].value_counts()`	`male 3, female 2`, Index = Werte
verschiedene Werte	`small["pclass"].nunique()`	`3`
Zeilen zählen	`len(small)`	`5`

Zeilen filtern

Ein Vergleich mit einer Spalte liefert eine Maske, eine Series aus `True` und `False`. In eckigen Klammern behält sie die Zeilen mit `True`.

Absicht	Code	Ergebnis
Maske bilden	`small["pclass"] == 3`	Series `False, False, True, True, False`
Zeilen auswählen	`small[small["pclass"] == 3]`	Zeilen mit den Etiketten 2 und 3

zwei Bedingungen	`erste = small["pclass"] == 1`
	`jung = small["age"] < 30`
	`small[erste & jung]`
		Zeilen 0 und 1

oder, nicht	`small[erste | jung]`, `small[~erste]`	Zeilen 0, 1, 3; Zeilen 2, 3, 4
Treffer zählen	`(small["sex"] == "male").sum()`	`3`
Maske mit Spalte	`small.loc[small["sex"] == "male", "age"]`	Series `0.75, NaN, 54.0`, Etiketten 1, 2, 4

Falle: `small[small["pclass"] == 1 & small["age"] < 30]` ohne Klammern bricht mit `TypeError` ab, weil `&` stärker bindet als `==` und `<`.

Etikett gegen Position: loc und iloc

`loc` wählt nach dem Etikett im Index, `iloc` nach der Position. Nach einem Filter unterscheiden sich beide.

Absicht	Code	Ergebnis
Filterergebnis merken	`sub = small[small["pclass"] == 3]`	Etiketten 2 und 3
Zeile nach Etikett	`sub.loc[2]`	Zeile mit Etikett 2
Zeile nach Position	`sub.iloc[0]`	dieselbe Zeile
fehlendes Etikett	`sub.loc[0]`	`KeyError`
eine Zelle	`small.loc[3, "age"]`	`18.0`
Bereich nach Etikett	`small.loc[1:3]`	Etiketten 1, 2, 3: drei Zeilen, Ende eingeschlossen
Bereich nach Position	`small.iloc[1:3]`	Positionen 1, 2: zwei Zeilen, Ende ausgeschlossen
neue Etiketten 0 bis N−1	`sub.reset_index(drop=True)`	liefert Tabelle mit Etiketten 0, 1

Falle: Eine Maske als Series wird über die Etiketten zugeordnet, nicht über die Reihenfolge; ein NumPy-Array als Maske über die Position.

Werte und Spalten ändern

Für Änderungen verwenden wir hier diese beiden Zuweisungsformen; die Methoden der Karte liefern dagegen neue Objekte. Alle Zeilen arbeiten auf der Kopie `beispiel`.

Absicht	Code	Ergebnis
Kopie zum Ausprobieren	`beispiel = small.copy()`	eigene Tabelle, `small` bleibt
ganze Spalte ersetzen oder anlegen	`beispiel["age_rounded"] = beispiel["age"].round()`	ändert `beispiel`, neue Spalte (weiter `float64`)
Datentyp einer Spalte	`beispiel["age"] = beispiel["age"].astype("float32")`	ändert `beispiel`
ausgewählte Zellen	`beispiel.loc[beispiel["pclass"] == 3, "age"] = 0.0`	ändert zwei Zellen
neues Objekt übernehmen	`beispiel = beispiel.drop(columns=["age_rounded"])`	ersetzt `beispiel`

Fehlerfall: `beispiel[beispiel["pclass"] == 3]["age"] = 0.0` ändert nichts und gibt eine Warnung aus. Der Filter liefert ein Zwischenergebnis, die Zuweisung schreibt dorthin (Copy-on-Write: ein Filterergebnis ist von seiner Quelle unabhängig). Richtig ist die `loc`-Form.

Falle: `beispiel["col"] = series` ordnet über die Etiketten zu, nicht über die Reihenfolge; ein Array oder eine Liste wird nach Position zugewiesen. Ein `NaN` in einer `int64`-Spalte macht die Spalte `float64`.

Text vereinheitlichen, Werte ersetzen

`.str` wendet Textmethoden auf jede Zelle einer Spalte an, ohne Schleife.

Absicht	Code	Ergebnis
Leerzeichen weg, klein	`small["sex"].str.strip().str.lower()`	liefert Series
Großbuchstaben	`small["sex"].str.upper()`	`FEMALE, MALE, …`
Textlänge je Zelle	`small["sex"].str.len()`	`6, 4, 4, 6, 4`
letztes Wort	`pd.Series(["PC 17599", "113803"]).str.split().str[-1]`	`17599, 113803`
Werte nach Zuordnung	`small["sex"].replace({"male": "m"})`	liefert Series
Werte unter Bedingung auf fehlend	`small["age"].mask(small["age"] < 1)`	`0.75` wird `NaN`

Fehlende Werte und Duplikate

In der Beispieltabelle werden Lücken als `NaN` dargestellt. Reduktionen wie `mean` und `sum` lassen sie aus; `count` zählt nur vorhandene Werte.

Absicht	Code	Ergebnis
Lücken erkennen	`small["age"].isna()`	Maske, `True` bei Etikett 2
Lücken zählen	`small["age"].isna().sum()`	`1`
vorhandene Werte zählen	`small["age"].count()`	`4` (gegen `len`: 5)
Mittelwert ohne Lücken	`small["age"].mean()`	`25.4375`
Zeilen mit Lücken in bestimmten Spalten weg	`small.dropna(subset=["age"])`	liefert 4 Zeilen
doppelte Zeilen zählen	`small.duplicated().sum()`	`0`
doppelte Zeilen entfernen	`small.drop_duplicates()`	liefert Tabelle

Falle: `dropna()` ohne `subset` entfernt jede Zeile mit irgendeiner Lücke, auch in Spalten, die nicht gebraucht werden. Duplikate erst nach dem Vereinheitlichen der Schreibweisen entfernen, sonst bleiben Paare stehen, die sich nur in der Schreibweise unterscheiden.

Gruppieren und zusammenfassen

`groupby` legt die Gruppen fest, erst `mean`, `sum`, `size` oder `count` fasst jede Gruppe zu einer Zahl zusammen. Bei den folgenden Beispielen mit einer ausgewählten Spalte entsteht eine Series mit den Gruppenschlüsseln als Index; ohne Spaltenauswahl entsteht ein DataFrame.

Absicht	Code	Ergebnis
Mittelwert je Gruppe	`small.groupby("sex")["age"].mean()`	`female 23.5, male 27.375`
Zeilen je Gruppe	`small.groupby("pclass").size()`	`1: 2, 2: 1, 3: 2`
vorhandene Werte je Gruppe	`small.groupby("pclass")["age"].count()`	`1: 2, 2: 1, 3: 1`
zwei Schlüssel	`je = small.groupby(["pclass", "sex"]).size()`	Series mit zweistufigem Index
als Tabelle	`je.unstack(fill_value=0)`	Zeilen `pclass`, Spalten `sex`
Häufigkeiten gegeneinander	`pd.crosstab(small["sex"], small["pclass"])`	Tabelle 2 × 3

Falle: Zeilen mit fehlendem Gruppenschlüssel lässt `groupby` weg; `dropna=False` legt für sie eine eigene Gruppe an. Der Mittelwert einer 0/1-Spalte ist der Anteil der Einsen, also eine Rate.

Verbinden, speichern, als Array übergeben

Für das Verbinden eine zweite Tabelle: `namen = pd.DataFrame({"pclass": [1, 2, 3], "klasse": ["Erste", "Zweite", "Dritte"]})`.

Absicht	Code	Ergebnis
Tabellen untereinander hängen	`pd.concat([small, small], ignore_index=True)`	liefert Tabelle mit 10 Zeilen, Etiketten 0 bis 9
Spalten über einen Schlüssel holen	`small.merge(namen, on="pclass", how="left")`	je Zeile von `small` die passende Zeile aus `namen`
als CSV schreiben	`small.to_csv("small.csv", index=False)`	Datei ohne Indexspalte
zurücklesen	`back = pd.read_csv("small.csv")`	DataFrame
vergleichen	`pd.testing.assert_frame_equal(back, small)`	still, wenn gleich; `check_dtype=False` ignoriert Typunterschiede
Zahlenspalten als Array	`small[["pclass", "age"]].to_numpy(dtype="float32")`	Array `(5, 2)`, `float32`
eine Spalte als Array	`small["sex"].to_numpy()`	Array `(5,)`

`how="left"` behält alle Zeilen von `small`, auch ohne Treffer; `validate="many_to_one"` bricht ab, wenn der Schlüssel in `namen` nicht eindeutig ist.
Falle: `to_numpy()` einer einzelnen Spalte kann eine schreibgeschützte Sicht auf die Tabelle sein; `to_numpy(copy=True)` liefert eine eigene, beschreibbare Kopie. Matplotlib (Figure und Axes) steht im Überblick des Workbooks; die eigene Referenzkarte Matplotlib kommt mit LE06.